# LUMO

## Workstream B: Open-LLM Livelihood Compiler

Project description, architecture and file-by-file guide

| Item | Detail |
|---|---|
| Purpose | Turn a street vendor's spoken or typed money story into a validated, structured financial state that the rest of LUMO can calculate on. |
| Package | LUMO_WorkstreamB.zip (Python 3.10+, FastAPI backend with a small browser demo page) |
| LLM provider | Sarvam AI only (model sarvam-105b) |
| Status | Working first version (MVP). 16 offline tests pass. Live Sarvam evaluation still to be run on your machine with your API key. |
| Owner | Workstream B (LLM / backend) |

### How to read this document

- **Section 1** explains the problem and what this workstream delivers.
- **Section 2** shows the architecture and who does what (LLM, code, user, other workstreams).
- **Section 3** explains which LLM is used, why, and with which settings.
- **Sections 4-6** explain the data schema, the rules and the API.
- **Section 7** describes every file in the zip in detail.
- **Sections 8-10** cover testing, running the project, limits and next steps.

# 1. Project description

LUMO is a financial 'livelihood twin' for street vendors and other unorganised workers in India. A vendor describes their situation in Hindi or Hinglish. LUMO builds a model of their cash, sales, costs, debts and goals, detects financial stress, stress-tests decisions (for example a 30% fall in sales for 14 days), and explains the result in plain language.

**Workstream B** is the first step in that chain: the **Livelihood Compiler**. Its job is to convert unstructured human language into structured facts, and to be honest about what it does not know. It is called a 'compiler' because it takes a messy source (a story) and produces a strict, checked output (a JSON state) or a clear error/question.

## The problem it solves

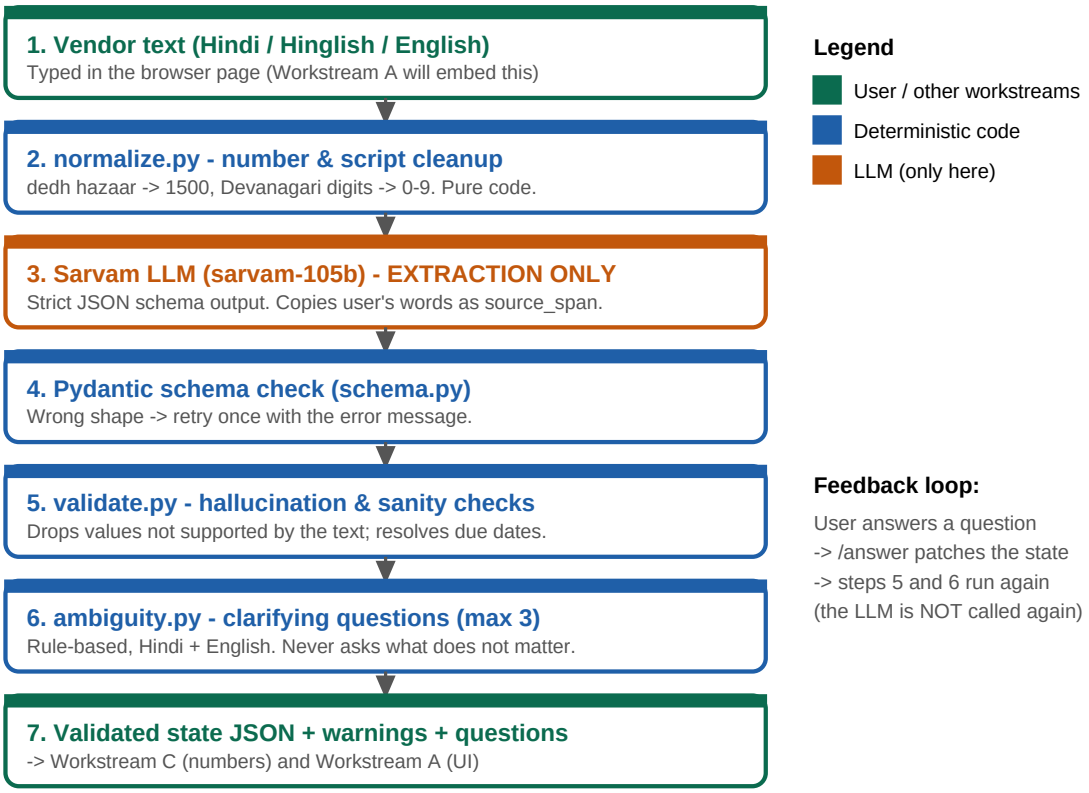
- People say things like 'main roz 1500 kamata hoon'. This could mean total sales or money left after buying stock. Treating the two as the same gives a completely wrong cash-flow picture, because **sales are not profit**.
- Language models can invent numbers. A financial tool must never do that.
- Dates are relative ('agle somvaar'), interest is vague ('3 rupaye sainkda' - per month? per year?), and amounts can conflict.

## What this workstream guarantees

| Requirement from the work plan | How it is met |
|---|---|
| Open-weight LLM extracts income, sales frequency, expenses, supplier debts, repayment dates, household obligations, goals | Sarvam sarvam-105b with a strict JSON schema (schema.py, prompts.py, llm.py) |
| Identify missing or ambiguous information and ask a follow-up when it matters | Rule engine in ambiguity.py asks at most 3 questions, only when the answer changes the calculation |
| Defined JSON schema and server-side validation | Pydantic models + validate.py checks |
| Every important value carries its source and confidence status | Each value has status (stated / inferred / ambiguous / missing), source_span (user's exact words) and confidence |
| Input correction flow | POST /answer patches any field and re-validates; the browser page shows answer buttons |
| Small test set | cases/cases.json (8 cases) + 16 offline unit tests |
| Definition of done: ambiguous inputs trigger clarification, not unsupported assumptions | Hallucination check drops unsupported numbers; unknown basis/period/interest period always becomes a question |

## 2. Architecture: who does what

The central design rule is: **the LLM only extracts and never decides**. Everything that must be exactly right (number parsing, dates, validation, which questions to ask) is ordinary Python code that behaves the same way every time.




### Responsibility table

| Actor | Does | Does NOT do |
|---|---|---|
| Sarvam LLM (sarvam-105b) | Reads the normalised text and fills the JSON schema. Copies the user's exact words as proof for each number. | No arithmetic, no totals, no profit, no date conversion, no guessing missing values, no choosing questions. |
| Python code (Workstream B) | Normalises numbers, validates the output, drops unsupported values, resolves dates, assigns confidence, generates questions, applies user corrections. | Does not call the LLM again after a correction. |
| The vendor (user) | Tells the story, answers clarifying questions, corrects wrong values. | - |
| Workstream A (frontend) | Builds the real UI. Calls /extract and /answer, shows questions and fact cards. | Should not parse the story itself. |
| Workstream C (calculations) | Reads the validated JSON and computes cash gaps, scenarios and savings deterministically. | Should not read raw text. |
| Workstream D (LUMO Samjho) | Can use the same Sarvam account for glossary explanations, translation and audio. | Not part of this package. |
| Workstream E (integration) | Uses /schema as the API contract; runs the benchmark with cases.json as a starting point. | - |

## 3. Which LLM and why

The project constraint is to use **only Sarvam AI**. The package therefore uses Sarvam's chat-completions API through the official `sarvamai` Python package, with the model `sarvam-105b` (configurable with the SARVAM_MODEL setting in .env).

### Why Sarvam is a good fit for this task

- **Indian-language focus:** Sarvam models are built for Indian languages, which matters because input is Hindi, Hinglish and sometimes Devanagari script.
- **Structured output support:** the chat API accepts a JSON-schema response format, so the model is forced to return the exact shape we need. This removes most formatting errors.
- **OpenAI-style chat interface:** system / user / assistant messages, so few-shot examples are easy to add.
- **Open-source positioning:** the work plan asks for an open-weight LLM. Confirm with your team that Sarvam's open-weight release meets that wording of the brief.

### Settings used and the reason for each

| Setting | Value | Why |
|---|---|---|
| model | sarvam-105b | Current Sarvam chat model available through the SDK (version 0.1.37 lists sarvam-105b and sarvam-105b-conversations). Older names such as sarvam-m and sarvam-30b are deprecated. |
| response_format | json_schema, strict = true | Forces valid JSON in our schema. If the API rejects it (HTTP 400/422), the client automatically falls back to plain json_object and Pydantic still enforces the schema. |
| temperature | 0.1 | Extraction should be repeatable, not creative. |
| reasoning_effort | None (off) | Thinking mode uses output tokens and adds delay; extraction does not need it. Reasoning tokens could also eat the max_tokens budget. |
| max_tokens | 4096 | Enough for a full state; a 'length' stop is treated as an error. |
| Retries | Backoff on 429/500/502/503/504; one self-correcting retry on invalid JSON | Hackathon-demo reliability. |
| Few-shot examples | 2 Hinglish examples | Show the model how to handle unknown income basis, unknown interest period and relative due dates. |

### Why the LLM is not trusted blindly

Even with strict JSON, a model can put a plausible-looking but invented number in a valid field. So every 'stated' value must come with `source_span`, the user's own words. Code then checks that the words really appear in the input and that the number appears inside those words. If either check fails, the value is dropped and a warning is raised.

# 4. Data model (the shared contract)

The schema lives in `lumo_compiler/schema.py` and is published at `GET /schema`. Workstreams A, C and E should build against it. Every field is required (nullable where unknown) so strict decoding works.

### The Fact object (used for every money value)

| Field | Meaning |
|---|---|
| value | A number in rupees (or a count/percentage), or null if unknown. |
| status | stated = user said it directly; inferred = obvious from context (rare); ambiguous = vague or unusual; missing = not said. |
| source_span | The user's exact words that support the value. 'user_correction' if the user typed the correction later. |
| confidence | Added by code, never by the LLM: stated 0.9, inferred 0.6, ambiguous 0.3, missing 0, user correction 1.0. |

### Top-level fields

| Field | Contents |
|---|---|
| cash_on_hand | Cash the vendor has right now (already available, not expected income). |
| min_reserve | Minimum cash the vendor wants to keep (user-defined). |
| income[] | kind (typical / good_day / bad_day / other), amount, basis (gross_sales / net_after_stock / unknown), period (day / week / month / unknown), days_per_week. |
| expenses[] | category (stock, transport, rent_fee, fuel, other), amount, period (day, week, month, one_time, unknown). |
| debts[] | creditor_type (supplier_credit, moneylender, bank_mfi, family, other), principal, interest_rate_pct, interest_period, installment, due_date_text. Code adds due_date. |
| household[] | description, amount, period (for example school fees). |
| goals[] | description, target_amount, target_date_text. Code adds target_date. |

### Output of /extract and /answer

```
{
  "schema_version": "0.1",
  "meta": { "today": "2026-10-20", "original_text": "...", "normalized_text": "..." },
  "extraction": { ...the fields above, each Fact with confidence... },
  "warnings":  [ {"code": "unsupported_value", "path": "cash_on_hand", "detail": "..."} ],
  "questions": [ {"id", "field_path", "question_en", "question_hi", "why_it_matters",
                  "answer_type": "choice|number|date|narrative", "options": [...] } ],
  "pending_questions_count": 0
}
```

# 5. Rules in detail

## 5.1 Validation (validate.py)

| Rule | What happens | Warning code |
|---|---|---|
| Value given but status is 'missing' | Value dropped | contradictory_fact |
| Negative value | Value dropped | negative_value |
| source_span not found in the user's text | Value dropped | unsupported_value |
| 'stated' value with no source_span, or number not inside the span | Value dropped | unsupported_value |
| Income outside a plausible range (day Rs.20-50,000; week Rs.100-3,50,000; month Rs.500-15,00,000) | Status set to 'ambiguous' and a confirm question is asked | suspicious_value |
| Installment larger than principal | Warning only | installment_exceeds_principal |
| Due date earlier than today | Warning only | due_date_in_past |

## 5.2 Clarifying questions (ambiguity.py)

Questions are asked only when the answer changes the calculation. The list is sorted by priority and cut to the top 3 so the vendor is not interrogated. Each question comes in Hindi (Devanagari) and English with a short 'why it matters' line.

| Trigger | Priority | Answer type |
|---|---|---|
| Income basis unknown (total sales or left after buying stock?) | 1 | choice |
| Income period unknown (day / week / month?) | 1 | choice |
| Interest rate given but interest period unknown | 1 | choice |
| Two different amounts for the same kind of income | 1 | narrative |
| No income mentioned at all | 1 | narrative |
| Cash on hand missing; income amount missing; debt amount missing; unusual amount to confirm; debt due date missing | 2 | number / date / choice |
| Days worked per week missing (when income is daily) | 4 | number |

## 5.3 Number and date normalisation

| Input | Becomes |
|---|---|
| dedh hazaar (Devanagari spelling also supported) | 1500 |
| do hazaar paanch sau | 2500 |
| sawa lakh | 125000 |
| 2k, 1,500, Devanagari digits | 2000, 1500, ASCII digits |
| agle somvaar (today = Tue 20 Oct 2026) | 2026-10-26 |
| 5 din mein / kal / parso / 25/10 | today+5 / tomorrow / today+2 / 25 Oct |

**Important:** the LLM receives the normalised text, and source spans are checked against that normalised text. This is why numbers like 'dedh hazaar' are safe: the model sees 1500, not a word it might misread.

## 5.4 Correction flow

When the vendor answers a question or edits a value, the browser calls `POST /answer` with a field path such as `income[0].basis`. The code applies the change, marks the value as stated by the user with confidence 1.0, then re-runs validation and question generation. **No new LLM call is made**, so the correction is instant and cannot be 'forgotten' by the model. Questions of type 'narrative' are answered by sending the extra sentence to /extract together with the original text.

## 6. API reference

| Endpoint | Request | Response / notes |
|---|---|---|
| GET / | - | Browser demo page. |
| GET /health | - | {ok: true, schema_version} |
| GET /schema | - | The self-contained JSON schema (no $ref) that LLM output must match. Use it as the API contract. |
| POST /extract | { text, today? } | Full state (extraction, warnings, questions). 502 if the model output is invalid after retries; 503 if Sarvam fails or the key is missing/rejected. |
| POST /answer | { state, field_path, value } | New state after applying the answer. 422 for an invalid answer (for example a basis value that is not allowed). |

### Example

```
POST /extract
{ "text": "Main roz 1500 kamata hoon. Abhi cash 800 hai." }

-> extraction.cash_on_hand.value = 800  (stated, span "cash 800")
   extraction.income[0].amount.value = 1500, basis = "unknown"
   questions[0]: "Is Rs.1,500 your total sales, or what is left after buying stock?"
                 options: gross_sales | net_after_stock

POST /answer { state, "income[0].basis", "net_after_stock" }
-> basis = "net_after_stock", question disappears
```

The Sarvam API key stays on the backend only (read from the .env file). It is never sent to the browser, as the work plan requires.

# 7. File-by-file description

## Folder layout

```
LUMO_WorkstreamB/
  setup.bat  run.bat  test.bat  setup.sh  run.sh
  requirements.txt  .env.example  README.md
  lumo_compiler/
    __init__.py  schema.py  normalize.py  dates.py  prompts.py  llm.py
    validate.py  ambiguity.py  patch.py  pipeline.py  api.py
    static/index.html
  cases/cases.json
  scripts/__init__.py  scripts/live_eval.py
  tests/test_normalize.py  tests/test_pipeline.py
```

## 7.1 Start-up and configuration files

| File | What it does |
|---|---|
| setup.bat | One-time Windows setup. Checks Python is installed, creates a .venv virtual environment, installs requirements.txt, creates .env from the example and opens it in Notepad so you can paste the Sarvam key. |
| run.bat | Starts the server (uvicorn) on port 8000 and opens http://127.0.0.1:8000 in the browser. |
| test.bat | Runs the offline tests (no key needed) and then the live Sarvam evaluation (needs the key). |
| setup.sh / run.sh | The same two steps for Mac or Linux. |
| requirements.txt | Python packages: sarvamai (Sarvam SDK), fastapi + uvicorn (web server), pydantic (validation), python-dotenv (reads .env), pytest + httpx (tests). |
| .env.example | Template for the secrets file. Copy to .env and set SARVAM_API_KEY. SARVAM_MODEL is optional (default sarvam-105b). |
| README.md | Short quick-start for developers. |

## 7.2 Core package: lumo_compiler/

| File | What it does | Key contents |
|---|---|---|
| schema.py | The data contract. Defines every field the LLM must output and builds the self-contained JSON schema sent to Sarvam. Rejects unknown fields (extra = forbid). | Fact, IncomeItem, ExpenseItem, DebtItem, HouseholdItem, GoalItem, Extraction, llm_json_schema(), SCHEMA_VERSION |
| normalize.py | Converts Hindi/Hinglish number words and Devanagari digits into plain numbers before the LLM sees the text. Also parses user-typed numeric answers. | normalize_text(), parse_number(); dictionaries for sau / hazaar / lakh, dedh / dhai / sawa, ek...pachas |
| dates.py | Turns relative due-date phrases into real dates, using a 'today' value passed in. Supports weekday names in Hindi and English, kal/parso/aaj, 'N din mein', dd/mm and ISO dates. | resolve_date_text() |
| prompts.py | The instructions given to the LLM: 9 strict rules (copy exact words, never calculate, use unknown when unsure, keep dates as text) plus two Hinglish few-shot examples that are also reused by tests. | SYSTEM_PROMPT, FEWSHOT, build_messages() |
| llm.py | The only file that talks to the network. Wraps the Sarvam SDK, requests strict JSON schema, retries on rate limits and server errors, falls back to json_object, and shows a clear message for a bad key. | SarvamExtractor class |
| validate.py | Server-side checks on the model output: unsupported value removal, range checks, due-date resolution, installment sanity and confidence assignment. | check_facts(), resolve_dates(), sanity_checks(), assign_confidence(), iter_facts() |
| ambiguity.py | Rule engine that decides which clarifying questions to ask, writes them in Hindi and English, attaches answer options and keeps the top 3. | build_questions(), CONFIRM |

| File | What it does | Key contents |
|---|---|---|
| patch.py | Applies a user answer or correction to a field by its path (for example debts[1].interest_period). Checks that enum answers are valid and marks the value as user-provided. | apply_patch(), ALLOWED |
| pipeline.py | Orchestrates everything. compile_text() normalises, calls the LLM, validates the JSON shape (one self-correcting retry) and finalises. finalize() is the deterministic post-processing that is re-run after every correction. | compile_text(), finalize(), ExtractionError |
| api.py | The FastAPI web server: serves the demo page and the /health, /schema, /extract and /answer endpoints. The LLM client is created lazily, so the server starts even before a key is set. | app, get_llm() |
| static/index.html | Self-contained demo page (no external libraries). Text box, three examples, Hindi/English question switch, answer buttons, fact table with status and confidence, warnings and a raw JSON viewer. Supports dark mode. | Calls /extract and /answer |
| __init__.py | Marks the folder as a Python package (empty). | - |

## 7.3 Tests and evaluation

| File | What it does |
|---|---|
| tests/test_normalize.py | Unit tests for number words, digit conversion, number parsing and date resolution (for example agle somvaar from Tue 20 Oct 2026 gives 26 Oct). |
| tests/test_pipeline.py | 12 pipeline tests using a fake LLM (so no key is needed): few-shot examples are valid; hallucinated numbers are dropped; mismatching spans are dropped; ambiguous income asks the basis question and the question disappears after answering; interest period is asked; due date is resolved; suspicious income triggers a confirm question; invalid JSON is retried; the API round trip works. |
| cases/cases.json | 8 fictional vendor inputs with expectations: ambiguous income, gross sales stated, relative due date, unknown interest period, Devanagari text, days per week, hallucination bait ('thoda bahut kama leta hoon') and no information at all. |
| scripts/live_eval.py | Runs cases.json against the real Sarvam API and prints PASS / FAIL per case. Checks values, fields, questions that must or must not be asked, and 'FALSE ASSUMPTION' if a number is invented. Run with: python -m scripts.live_eval |

## 8. Walk-through of one request

Input: *Supplier ko 5000 dena hai agle somvaar. Mahajan se 10000 liya hai, 3 rupaye sainkda.* (today is Tuesday 20 Oct 2026)

| Step | What happens |
|---|---|
| 1 normalize.py | No number words to convert; text is passed on unchanged. |
| 2 Sarvam | Returns two debts. Supplier: principal 5000 (span '5000 dena hai'), due_date_text 'agle somvaar'. Lender: principal 10000, interest_rate_pct 3 (span '3 rupaye sainkda'), interest_period 'unknown'. |
| 3 Pydantic | JSON matches the schema, so no retry is needed. |
| 4 validate.py | All spans are found in the text and match their numbers. 'agle somvaar' becomes 2026-10-26. |
| 5 ambiguity.py | Asks: is the 3% charged per day, week, month or year? Asks the lender's due date. Also asks about missing income and cash (top 3 shown, the rest counted in pending_questions_count). |
| 6 Output | State JSON for Workstream C plus the questions for the UI. |

## 9. How to run

- Unzip LUMO_WorkstreamB.zip (Extract All). Open the extracted folder.
- Double-click **setup.bat**. Paste your Sarvam API key into the .env file that opens, save and close.
- Double-click **run.bat**. The demo page opens in your browser. Click 'Example 1', then Extract.
- Optional: double-click **test.bat** for the offline tests and the live Sarvam evaluation.
- Manual alternative: python -m venv .venv, activate it, pip install -r requirements.txt, then python -m uvicorn lumo_compiler.api:app --port 8000.

## 10. Current limits and next steps

| Item | Status / plan |
|---|---|
| Live Sarvam run | Not yet tested with a real key. Run test.bat; any FAIL lines show what to tune in prompts.py. |
| Test set size | 8 cases now; the work plan asks for a small benchmark (about 25-30 recommended). Add incomplete records, conflicting amounts, mixed scripts and income shocks. |
| Multi-turn merge | A new message is re-extracted together with the old text. A proper merge into an existing state is a future improvement. |
| Number words | Common patterns are handled (dedh, dhai, sawa, sau, hazaar, lakh). Rare forms such as 'saadhe teen hazaar' are not yet covered. |
| Languages | Hindi, Hinglish and English. Punjabi can be added in Workstream D first, then tested here. |
| Security | CORS is open for development. Restrict it and add rate limiting before public deployment. |
| Model wording | Confirm that Sarvam's open-weight status satisfies the 'open-weight LLM' wording of the work plan. |